

Universidad Autónoma de Santo Domingo

PRIMADA DE AMÉRICA / Fundada el 28 de octubre de 1538

JOANFI ESPINAL LOPEZ

100519996

Lab Lenguaje de Programación III

INF5170-1

MVC en el desarrollo web moderno: origen, evolución y uso actual en aplicaciones web

Radhames Silverio Gonzalez

MVC en el desarrollo web moderno: origen, evolución y uso actual en aplicaciones web

Introducción

El desarrollo de aplicaciones web requiere organizar el código de forma que los datos, la presentación y las acciones del usuario puedan mantenerse sin crear dependencias innecesarias. Cuando toda la lógica se coloca en un mismo archivo, cualquier cambio puede afectar otras partes del sistema y dificultar las pruebas, la corrección de errores y el trabajo en equipo. Para responder a este tipo de problemas se utilizan patrones de arquitectura y diseño, entre ellos el Modelo-Vista-Controlador, conocido como MVC.

MVC propone dividir una aplicación en tres responsabilidades principales: el Modelo, que representa los datos y las reglas del negocio; la Vista, que presenta la información; y el Controlador, que interpreta las solicitudes y coordina la respuesta. Aunque sus ideas se originaron hace varias décadas, el patrón continúa presente en frameworks utilizados actualmente para crear sitios web, sistemas empresariales y servicios que ofrecen datos a otros programas (Microsoft, s. f.; Spring, s. f.).

El propósito de esta investigación es explicar el origen y la finalidad de MVC, describir sus componentes, analizar sus ventajas y limitaciones, y comparar su aplicación tradicional con la de frameworks modernos. También se presenta un ejemplo de registro de estudiantes para mostrar cómo se distribuyen las responsabilidades en un caso práctico.

1. ¿Qué es el patrón MVC?

El Modelo-Vista-Controlador (MVC) es un patrón arquitectónico que organiza una aplicación en tres componentes relacionados, pero con responsabilidades diferenciadas. Su objetivo principal es separar las preocupaciones del sistema: la gestión de los datos y las reglas, la presentación visual y el manejo de las interacciones o solicitudes. Esta separación permite modificar una parte de la aplicación con menos necesidad de alterar las demás.

En una aplicación web típica, el navegador envía una solicitud al servidor. El sistema de rutas la dirige al controlador correspondiente; este interpreta la solicitud y consulta o modifica el modelo. Después, puede devolver una vista HTML o una respuesta en otro formato, como JSON. El flujo concreto depende del framework y del tipo de aplicación, por lo que MVC debe entenderse como una guía de organización y no como una secuencia rígida que todos los sistemas ejecutan exactamente igual (Microsoft, s. f.).

MVC no es un lenguaje de programación ni una herramienta específica. Es un patrón que puede implementarse con diferentes lenguajes y frameworks. Por ejemplo, puede utilizarse con Java y Spring MVC, C# y ASP.NET Core MVC, PHP y Laravel, o Ruby y Ruby on Rails.

2. Origen del modelo MVC y propósito inicial

El origen de MVC se sitúa en los trabajos de Trygve Reenskaug durante su estancia en Xerox Palo Alto Research Center (PARC), a finales de la década de 1970. En 1979, Reenskaug documentó ideas que evolucionaron desde “Thing-Model-View-Editor”

hasta los términos Modelo, Vista y Controlador. La propuesta estaba relacionada con la construcción de interfaces interactivas, especialmente en el contexto de Smalltalk, y no surgió originalmente como una arquitectura exclusiva para páginas web (Reenskaug, 1979).

Su propósito era ayudar a las personas a trabajar con información compleja desde una interfaz, manteniendo diferenciados los datos que representaban el problema y las distintas maneras de mostrarlos o manipularlos. Por ejemplo, una misma información podía necesitar presentaciones diferentes sin tener que duplicar toda la lógica que la gestionaba.

Con el crecimiento de las aplicaciones cliente-servidor y posteriormente de la web, el patrón se adaptó a nuevas necesidades. Los frameworks comenzaron a utilizar controladores para recibir solicitudes HTTP, modelos para representar y persistir información, y vistas para producir páginas dinámicas. Por eso, el MVC utilizado en el desarrollo web moderno conserva la idea original de separación, aunque sus mecanismos concretos son diferentes de los sistemas de interfaces gráficas para los que se concibió.

3. Componentes del modelo MVC

3.1. Modelo (Model)

El Modelo representa los datos y las reglas principales del dominio de la aplicación. Puede incluir entidades, validaciones, operaciones de negocio y mecanismos para consultar o guardar información en una base de datos. En un sistema de estudiantes, por ejemplo, el modelo puede representar matrícula, nombre, carrera y correo electrónico, además de comprobar que la matrícula no se repita y que el correo tenga un formato válido.

El modelo no debería depender de cómo se presenta la información en pantalla. Así, una misma lógica puede utilizarse desde una página web, una aplicación móvil o una API. En proyectos grandes, las reglas de negocio también pueden organizarse en servicios o capas de dominio, en lugar de concentrarlas todas dentro de una sola clase de modelo.

3.2. Vista (View)

La Vista es la parte encargada de presentar la información al usuario. En aplicaciones web puede ser una plantilla HTML que muestra un formulario, una tabla de resultados, mensajes de confirmación o errores de validación. La vista recibe los datos que necesita y los representa de forma comprensible.

La vista debe evitar contener reglas de negocio complejas. Su función principal es presentar la información y ofrecer los elementos con los que el usuario interactúa. Separarla del modelo facilita cambiar el diseño, los textos o la estructura de una página sin reescribir las reglas que controlan los datos.

3.3. Controlador (Controller)

El Controlador recibe y procesa las solicitudes o acciones del usuario. En una aplicación web, normalmente se relaciona con una ruta o un conjunto de rutas. Puede leer los datos

enviados por un formulario, solicitar al modelo que realice una operación, coordinar validaciones y decidir qué respuesta devolver.

El controlador no debe convertirse en el lugar donde se acumula toda la lógica del sistema. Si contiene consultas, reglas, validaciones y generación de páginas en exceso, se vuelve difícil de mantener. Una buena implementación mantiene el controlador enfocado en coordinar el flujo y delega el trabajo especializado a modelos o servicios.

4. Ventajas y desventajas de MVC

Ventajas

Separación de responsabilidades: cada componente tiene una función más clara.

Mantenimiento: los cambios en la presentación pueden realizarse sin modificar necesariamente las reglas del negocio.

Reutilización: la lógica del modelo puede servir a distintas vistas o tipos de cliente.

Pruebas: es posible probar partes del sistema de manera más independiente.

Trabajo en equipo: diferentes personas pueden trabajar en modelos, vistas y controladores con acuerdos definidos.

Organización: la estructura facilita ubicar archivos y comprender el recorrido de una solicitud.

Desventajas

Complejidad inicial: en proyectos pequeños puede parecer excesivo crear varios archivos y componentes.

Curva de aprendizaje: el desarrollador debe comprender cómo se comunican las partes y cómo trabaja el framework.

Controladores demasiado grandes: si se diseña mal, la lógica puede terminar concentrada en ellos.

Variaciones entre frameworks: no todos interpretan MVC exactamente de la misma manera.

No resuelve todos los problemas arquitectónicos: por sí solo no define seguridad, despliegue, escalabilidad, comunicación entre servicios ni organización completa de sistemas distribuidos.

En consecuencia, MVC no garantiza automáticamente que una aplicación esté bien diseñada. Sus beneficios dependen de que las responsabilidades se distribuyan correctamente y de que se combine con prácticas como validación, pruebas, control de acceso y manejo seguro de errores.

5. ¿Por qué MVC sigue siendo importante en la programación actual?

MVC continúa siendo importante porque la separación de responsabilidades sigue siendo útil independientemente de la edad del patrón. Las aplicaciones modernas todavía necesitan recibir solicitudes, aplicar reglas, consultar datos y producir respuestas. MVC ofrece un vocabulario común para organizar esas tareas y facilita que los desarrolladores entiendan proyectos creados por otras personas.

También sirve como base para aprender frameworks. Conceptos como rutas, controladores, modelos, plantillas, validación y respuestas HTTP se comprenden mejor cuando se conoce el propósito de cada componente. Esta base permite distinguir entre el patrón arquitectónico y las herramientas específicas que lo implementan.

Decir que MVC es antiguo no significa que esté obsoleto. Significa que es un patrón establecido y ampliamente conocido. Sin embargo, no es la única alternativa: algunas aplicaciones usan arquitecturas de componentes, arquitecturas hexagonales, servicios independientes o enfoques orientados a eventos. En proyectos reales pueden combinarse varias ideas, utilizando MVC en la capa web y otras estructuras para organizar la lógica interna.

6. Uso de MVC en frameworks modernos

Los frameworks actuales automatizan muchas tareas que antes debían programarse manualmente. Entre ellas se encuentran el enrutamiento de solicitudes, la conversión de datos enviados por formularios, la validación, la autenticación, la generación de vistas, la inyección de dependencias y la creación de respuestas HTTP. El desarrollador sigue trabajando con responsabilidades similares a las de MVC, pero dispone de convenciones y herramientas que reducen el código repetitivo.

En las aplicaciones modernas, el controlador puede devolver una página HTML, una redirección, un archivo o datos en JSON. Por esta razón, MVC no se limita a sitios web tradicionales: también puede utilizarse para construir APIs que consumen aplicaciones móviles, interfaces JavaScript u otros servicios. En estos casos, la respuesta no siempre incluye una vista HTML; el controlador puede devolver datos serializados.

Es importante señalar que los frameworks no implementan MVC de forma idéntica. Algunos se acercan más a la separación clásica, mientras que otros emplean nombres diferentes o añaden componentes como servicios, repositorios, middleware y capas de presentación. Lo fundamental es entender las responsabilidades y no memorizar únicamente nombres de carpetas.

7. Comparación de MVC tradicional y frameworks actuales

El MVC tradicional describe una separación conceptual entre Modelo, Vista y Controlador. En las aplicaciones web modernas, cada framework adapta esa idea a su lenguaje, convenciones y herramientas. La siguiente tabla resume diferencias importantes.

Framework	Cómo organiza la arquitectura	Aspecto distintivo
Spring Web MVC (Java)	Usa controladores que reciben solicitudes, modelos para los datos y vistas para presentar respuestas.	Incluye DispatcherServlet, controladores anotados, conversión de mensajes y soporte para respuestas HTML o APIs.
ASP.NET Core MVC (C#)	Separa modelos, vistas Razor y controladores con acciones.	Ofrece model binding, validación, inyección de dependencias, filtros y soporte para aplicaciones web y APIs.
Laravel (PHP)	Organiza rutas, modelos Eloquent, controladores y plantillas Blade.	Usa convenciones y herramientas integradas para bases de datos, validación, rutas y vistas.
Django (Python)	Utiliza una arquitectura denominada Model-Template-View (MTV).	El modelo gestiona datos, la plantilla presenta contenido y la vista de Django procesa la solicitud; esta última cumple parte del papel que en MVC se atribuye al controlador.
Ruby on Rails (Ruby)	Organiza modelos, vistas y controladores, con convenciones que reducen configuración repetitiva.	Active Record facilita el trabajo con datos y las convenciones permiten generar componentes de una aplicación rápidamente.

7.1. Spring Web MVC

Spring Web MVC forma parte del ecosistema Spring para Java y se apoya en la API de Servlets. El DispatcherServlet funciona como un componente central que recibe las solicitudes y coordina su procesamiento. Los controladores pueden definirse mediante anotaciones y devolver vistas o datos, según la necesidad de la aplicación (Spring, s. f.).

7.2. ASP.NET Core MVC

ASP.NET Core MVC implementa el patrón para crear aplicaciones web y APIs. Las solicitudes se dirigen a controladores y acciones; el framework ayuda a enlazar los datos de la solicitud con parámetros u objetos, validar entradas y generar respuestas. Las

vistas Razor permiten combinar HTML con expresiones de C# para presentar datos (Microsoft, s. f.).

7.3. Laravel

Laravel es un framework de PHP que organiza la aplicación mediante rutas, controladores, modelos y vistas. Las rutas definen qué acción responde a una dirección; los controladores coordinan el procesamiento; los modelos suelen interactuar con la base de datos mediante Eloquent; y Blade permite crear plantillas de presentación. Laravel automatiza gran parte de esta conexión, aunque el programador debe mantener cada componente enfocado en su responsabilidad (Laravel, s. f.).

7.4. Django

Django utiliza el nombre Model-Template-View (MTV), por lo que no coincide literalmente con los nombres de MVC. En términos generales, el modelo representa los datos, la plantilla define la presentación y la vista de Django procesa la solicitud y devuelve una respuesta. Esta diferencia es principalmente de terminología y distribución de responsabilidades; por ello, al comparar frameworks es necesario estudiar cómo fluye una solicitud en cada uno, en lugar de asumir que todos usan los mismos nombres.

7.5. Ruby on Rails

Ruby on Rails aplica MVC y promueve convenciones para organizar el código. Sus modelos suelen trabajar con la base de datos mediante Active Record, los controladores gestionan las solicitudes y las vistas producen respuestas en formatos como HTML o JSON. Las guías oficiales de Rails presentan MVC como una de las bases para comprender la estructura de sus aplicaciones (Ruby on Rails, s. f.).

Diferencias generales

En la práctica, los frameworks modernos conservan la idea de separación, pero añaden herramientas que el patrón original no especificaba. Por ejemplo, la inyección de dependencias facilita proporcionar servicios a los controladores; el enrutamiento conecta URLs con acciones; los motores de plantillas generan páginas; y los serializadores transforman objetos en JSON. MVC no exige por sí solo estas herramientas: cada framework las incorpora según sus objetivos.

Por tanto, elegir un framework no consiste únicamente en decidir si utiliza MVC. También deben considerarse el lenguaje, el tamaño del proyecto, las necesidades del equipo, la seguridad, el rendimiento, la comunidad, el mantenimiento y la integración con otros sistemas.

8. Ejemplo práctico: sistema de registro de estudiantes

Supóngase que una universidad necesita una aplicación para registrar estudiantes. El formulario solicita matrícula, nombre, carrera y correo electrónico. Cuando el usuario presiona el botón “Registrar”, el sistema debe validar los datos, guardar la información y mostrar una confirmación o los errores encontrados.

Modelo: Estudiante

El modelo Estudiante representa los datos del estudiante y las reglas relacionadas con ellos. Sus atributos principales pueden ser matrícula, nombre, carrera y correo. Entre sus responsabilidades están verificar que los campos obligatorios tengan contenido, comprobar que el correo tenga un formato válido y colaborar con la capa de persistencia para guardar o recuperar registros. La validación definitiva debe realizarse en el servidor, aunque el formulario también puede incluir validaciones para ayudar al usuario.

Vista: formulario de registro

La vista muestra los campos de matrícula, nombre, carrera y correo, junto con el botón "Registrar". También puede presentar mensajes como "El estudiante se registró correctamente" o "La matrícula ya existe". La vista se ocupa de la presentación; no debería conectarse directamente a la base de datos ni contener las reglas principales del negocio.

Controlador: EstudianteController

El controlador recibe la solicitud enviada desde el formulario. Comprueba que los datos recibidos sean válidos, coordina la operación de registro con el modelo o con un servicio y selecciona la respuesta. Si el registro es correcto, puede devolver una vista de confirmación o redirigir a la lista de estudiantes. Si hay errores, puede volver a mostrar el formulario con mensajes comprensibles.

Flujo de funcionamiento

El usuario abre la página de registro y el servidor presenta la vista del formulario.

El usuario completa los datos y envía el formulario.

La ruta de la aplicación dirige la solicitud al controlador de estudiantes.

El controlador valida la entrada y solicita registrar al estudiante.

El modelo y la capa de persistencia aplican las reglas necesarias y guardan los datos si son correctos.

El controlador devuelve una confirmación o presenta los errores en la vista.

Pseudocódigo ilustrativo

```
MODELO: Estudiante
atributos: matricula, nombre, carrera, correo
validar():
comprobar campos obligatorios
comprobar formato del correo
comprobar que la matrícula no esté registrada

CONTROLADOR: EstudianteController
registrar(solicitud):
datos = leer datos del formulario
```

resultado = Estudiante.validar(datos)
si resultado es válido:
guardar estudiante
devolver vista de confirmación
si no:
devolver formulario con errores

VISTA: formulario_estudiante.html
mostrar campos de matrícula, nombre, carrera y correo
mostrar errores, si existen
mostrar botón "Registrar"

Este ejemplo es conceptual y no corresponde a un framework específico. En un proyecto real, el guardado en la base de datos podría estar a cargo de un repositorio o servicio, y la validación podría distribuirse entre el modelo, una clase de validación y el controlador. Lo importante es que la presentación no se encargue de guardar directamente los datos y que el controlador coordine el proceso sin asumir todas las responsabilidades.

9. Conclusión personal

En mi opinión, comprender MVC es importante antes de trabajar con frameworks modernos porque permite entender la estructura de una aplicación y no limitarse a seguir instrucciones o copiar código. Cuando se conoce la función del modelo, la vista y el controlador, resulta más sencillo localizar errores, organizar archivos y saber en qué parte del proyecto debe realizarse cada cambio.

También considero que MVC demuestra que una buena arquitectura no depende solamente de utilizar la tecnología más nueva, sino de organizar el software de manera clara. Aunque el patrón surgió hace varias décadas y hoy existen otras formas de construir sistemas, sus principios continúan presentes en herramientas como Spring MVC, ASP.NET Core MVC, Laravel y Ruby on Rails. Incluso Django, que utiliza la denominación MTV, comparte la idea de separar los datos, la presentación y el procesamiento de solicitudes.

Por último, aprender MVC proporciona una base para estudiar arquitecturas más avanzadas y comprender cómo se conectan las páginas, los servicios y las bases de datos. No significa que todas las aplicaciones deban construirse con MVC, sino que conocerlo ayuda a tomar decisiones mejor fundamentadas y a mantener el código más ordenado y fácil de probar.

Referencias

Microsoft. (s. f.). Overview of ASP.NET Core MVC. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/mvc/overview

Reenskaug, T. (1979). Models–Views–Controllers. Xerox PARC. https://folk.universitetetioslo.no/trygver/1979/mvc-2/1979-12-MVC.pdf

Ruby on Rails. (s. f.). Getting started with Rails. https://guides.rubyonrails.org/getting_started.html

Spring. (s. f.). Spring Web MVC. Spring Framework Reference Documentation. https://docs.spring.io/spring/reference/web/webmvc.html